

前向-后向算法计算条件概率 $P(y | x)$

给定观测序列 $x = (x_1, \dots, x_n)$ ，计算算配分函数 $Z(x)$ 以及标签序列 y 的条件概率 $P(y | x)$ 。

对一条完整的标签 $y = (y_1, \dots, y_n)$ ，其未归一化得分定义为：

$$\Psi(y, x) = \exp \left(\sum_{i=1}^n s_i(y_i, x) + \sum_{i=2}^n t(y_{i-1}, y_i) \right) \quad (1)$$

其中：

- $s_i(y, x) = \sum_k w_k f_k(y, x, i)$ 是状态得分；
- $t(y', y) = T[y', y]$ 是转移得分。

条件概率的定义就是：

$$P(y | x) = \frac{\Psi(y, x)}{Z(x)}, \quad Z(x) = \sum_{y'} \Psi(y', x) \quad (2)$$

难点在于 $Z(x)$ 是对所有可能标签序列求和，标签序列数量是 L^n (L 为标签种类数)，直接枚举是指数级的，这正是需要前向-后向算法的原因。

定义（非对数形式，与代码的对数形式等价）

$$\begin{aligned} \alpha_i(y) &= \exp(s_1(y, x)), \quad \alpha_i(y) = \exp(s_i(y, x)) \sum_{y'} \alpha_{i-1}(y') \exp(t(y', y)), \quad i = 2, \dots, n \\ \beta_n(y) &= 1, \quad \beta_i(y) = \sum_{y'} \exp(t(y, y')) \exp(s_{i+1}(y', x)) \beta_{i+1}(y'), i = n-1, n-2, \dots, 1 \end{aligned} \quad (3)$$

注意，这里 $\alpha_i(y)$ 的自变量 y 是一维变量，取值范围为标签集合。

命题

$\alpha_i(y)$ 等于所有"长度为 i 、且第 i 位标签固定为 y "的部分路径的未归一化得分之和：

$$\alpha_i(y) = \sum_{y_1, \dots, y_{i-1}} \exp \left(\sum_{k=1}^i s_k(y_k, x) + \sum_{k=2}^i t(y_{k-1}, y_k) \right) \Big|_{y_i=y} \quad (4)$$

归纳证明

基例 ($i = 1$)：路径长度为 1，没有转移项，直接就是 $\alpha_1(y) = \exp(s_1(y, x))$ ，与定义一致。

归纳步（假设 $\alpha_{i-1}(y')$ 成立，证明 $\alpha_i(y)$ ）：

长度为 i 且第 i 位为 y 的部分路径，等价于"长度为 $i-1$ 的部分路径（任意末位 y' ）+ 从 y' 转移到 y + 在位置 i 取标签 y "的组合。把这些路径的得分按前 $i-1$ 位的取值求和、再乘上第 i 位新增的转移项和状态项：

$$\alpha_i(y) = \sum_{y'} \left[\sum_{y_1, \dots, y_{i-2}} \exp \left(\sum_{k=1}^{i-1} s_k(y_k, x) + \sum_{k=2}^{i-1} t(y_{k-1}, y_k) \right) \Big|_{y_{i-1}=y'} \right] \exp(t(y', y)) \exp(s_i(y, x)) \quad (5)$$

方括号内正是归纳假设中的 $\alpha_{i-1}(y')$ ，代入即得：

$$\alpha_i(y) = \exp(s_i(y, x)) \sum_{y'} \alpha_{i-1}(y') \exp(t(y', y)) \quad (6)$$

与定义完全一致，归纳成立。■

这与代码中的递推（对数空间下用 `logsumexp` 代替 $\sum$ ，`exp` 隐含在对数里）完全对应：

```
alpha[i] = node_sc[i] + self.logsumexp(alpha[i-1][:, None] + self.T, axis=0)
```

用 $\alpha_n(y)$ 算出 $Z(x)$ 和条件概率

由上面命题，取 $i = n$ ，那么 $\alpha_n(y)$ 就是所有以 $y_n = y$ 结尾的完整路径的得分之和。对 y 求和即覆盖了全部可能的完整路径，正好是 $Z(x)$ 的定义：

$$Z(x) = \sum_y \Psi(y, x) = \sum_{y_n} \alpha_n(y_n) \quad (7)$$

对应代码（对数空间）：

```
z = self.logsumexp(alpha[-1], axis=0)
```

这一步是关键：本来需要枚举 L^n 条路径才能算的 $Z(x)$ ，现在只需要做 n 步、每步 $O(L^2)$ 的递推（动态规划），复杂度从指数级降到了 $O(nL^2)$ 。

有了 $Z(x) = \sum_y \alpha_n(y)$ ，任意给定的完整标签序列 $\hat{y} = (\hat{y}_1, \dots, \hat{y}_n)$ 的条件概率就是：

$$P(\hat{y} | x) = \frac{\Psi(\hat{y}, x)}{Z(x)} = \frac{\exp(\sum_i s_i(\hat{y}_i, x) + \sum_i t(\hat{y}_{i-1}, \hat{y}_i))}{\sum_{y_n} \alpha_n(y_n)} \quad (8)$$

- 分子 $\Psi(\hat{y}, x)$ 是给定路径按定义直接算出来的（不需要 α ）；
- 分母 $Z(x)$ 则依赖 $\alpha_n(y)$ 才能高效算出——这正是 α 在计算 $P(y|x)$ 中扮演的角色：它不直接给出某条特定路径的概率，而是提供归一化所需的 $Z(x)$ 。

对应代码中 `neg_log_likelihood_and_grad` 里：

```
gold_score = node_sc[np.arange(n), tag_ids].sum() + sum(
    self.T[tag_ids[i], tag_ids[i + 1]] for i in range(n - 1)
) # 这就是 log Ψ(ŷ, x)
nll = z - gold_score      # -log P(ŷ|x) = log Z(x) - log Ψ(ŷ,x)
```

即：

$$-\log P(\hat{y} | x) = \log Z(x) - \log \Psi(\hat{y}, x) \quad (9)$$

与上面推导的公式完全一致。

损失函数：负对数似然

对训练样本 (x, y) ，条件概率为 $P(y | x) = \Psi(y, x)/Z(x)$ 。定义负对数似然作为损失：

$$\text{NLL}(w) = -\log P(y | x) = \log Z(x) - \log \Psi(y, x) \quad (10)$$

把 $\log \Psi(y, x)$ 展开（记 $F_k(x, y) = \sum_i f_k(y_{i-1}, y_i, x, i)$ 为路径 y 上特征 k 的总激活次数）：

$$\text{NLL}(w) = \log Z(x) - \sum_k w_k F_k(x, y) \quad (11)$$

这与代码中的写法完全对应：

```
gold_score = node_sc[np.arange(n), tag_ids].sum() + sum(
    self.T[tag_ids[i], tag_ids[i + 1]] for i in range(n - 1)
)
nll = z - gold_score    # nll = log Z(x) - log Ψ(y*, x)
```

梯度的一般形式：期望特征计数 - 经验特征计数

对任意权重 w_k 求偏导：

$$\frac{\partial \text{NLL}}{\partial w_k} = \frac{\partial \log Z(x)}{\partial w_k} - F_k(x, y) \quad (12)$$

关键是算 $\frac{\partial \log Z(x)}{\partial w_k}$ 。由 $Z(x) = \sum_y \exp(\sum_k w_k F_k(x, y))$ ：

$$\frac{\partial \log Z(x)}{\partial w_k} = \frac{1}{Z(x)} \sum_y \exp\left(\sum_{k'} w_{k'} F_{k'}(x, y)\right) F_k(x, y) = \sum_y P(y | x) F_k(x, y) = \mathbb{E}_{P(y|x)}[F_k(x, y)] \quad (13)$$

因此：

$$\boxed{\frac{\partial \text{NLL}}{\partial w_k} = \mathbb{E}_{P(y|x)}[F_k(x, y)] - F_k(x, y)} \quad (14)$$

即“模型预测下的期望特征计数” - “训练数据给出的经验特征计数”。这与前一篇文档提到的对数似然梯度公式（经验 - 期望）互为相反数——因为这里对负对数似然求导，符号自然翻转。

由于 $F_k(x, y) = \sum_i f_k(y_{i-1}, y_i, x, i)$ 对 i 线性可加，期望也按位置拆开：

$$\mathbb{E}_{P(y|x)}[F_k(x, y)] = \sum_{i=1}^n \mathbb{E}_{P(y|x)}[f_k(y_{i-1}, y_i, x, i)] = \sum_{i=1}^n \sum_{a,b} P(y_{i-1}=a, y_i=b | x) f_k(a, b, x, i) \quad (15)$$

这一步把“整条路径上的期望”转化成了“逐位置的边缘概率加权求和”——而边缘概率正是前向-后向算法的计算目标。状态特征（只依赖 y_i ）可以看作转移特征在忽略 y_{i-1} 后的特例，因此下面统一先推导边缘概率的公式，再分别给出状态权重和转移权重的梯度。

用 α 、 β 计算边缘概率

单点边缘概率 $P(y_i = y | x)$

$\alpha_i(y)$ 是“第 i 位为 y 的所有前缀路径得分之和”， $\beta_i(y)$ 按定义是“从位置 $i+1$ 到 n 、衔接在 $y_i = y$ 之后的所有后缀路径得分之和”（不含位置 i 自身的状态得分，因为 $s_i(y, x)$ 已经包含在 $\alpha_i(y)$ 里）。两者相乘，恰好覆盖了全部满足 $y_i = y$ 的完整路径得分：

$$\alpha_i(y) \cdot \beta_i(y) = \sum_{y_1, \dots, y_{i-1}, y_{i+1}, \dots, y_n} \Psi(y, x) \Big|_{y_i=y} \quad (16)$$

归一化后即得单点边缘概率：

$$\boxed{P(y_i = y | x) = \frac{\alpha_i(y) \beta_i(y)}{Z(x)}} \quad (17)$$

对应代码：

```
marg = np.exp(alpha + beta - z)    # marg[i, y] = P(y_i = y | x)
```

（代码中 `alpha`、`beta` 存的是对数值，所以这里用加法再 `exp` 还原，等价于上式的乘法。）

相邻两点边缘概率 $P(y_{i-1}=a, y_i=b \mid x)$

这是转移特征梯度所必需的量，必须结合后向算法才能算出。

命题：

$$P(y_{i-1}=a, y_i=b \mid x) = \frac{\alpha_{i-1}(a) \cdot \exp(t(a, b) + s_i(b, x)) \cdot \beta_i(b)}{Z(x)} \quad (18)$$

证明：固定 $y_{i-1} = a, y_i = b$ ，对其余位置 $y_1, \dots, y_{i-2}$ 和 $y_{i+1}, \dots, y_n$ 求和，把整条路径的势函数按位置 $i-1$ 与 i 之间的连接拆成三段：

$$\begin{aligned} \sum_{\substack{y_1, \dots, y_{i-2} \\ y_{i+1}, \dots, y_n}} \Psi(y, x) \Big|_{y_{i-1}=a, y_i=b} &= \underbrace{\left[\sum_{y_1, \dots, y_{i-2}} \exp\left(\sum_{k \leq i-1} s_k(y_k, x) + \sum_{k \leq i-1} t(y_{k-1}, y_k)\right) \right]_{y_{i-1}=a}}_{= \alpha_{i-1}(a)} \\ &\cdot \underbrace{\exp(t(a, b)) \cdot \exp(s_i(b, x))}_{\text{连接 } i-1 \rightarrow i \text{ 的边}} \cdot \underbrace{\left[\sum_{y_{i+1}, \dots, y_n} \exp\left(\sum_{k > i} s_k(y_k, x) + \sum_{k > i} t(y_{k-1}, y_k)\right) \right]_{y_i=b}}_{= \beta_i(b)} \end{aligned} \quad (19)$$

三段分别正是 $\alpha_{i-1}(a)$ 、连接边的势函数、以及 $\beta_i(b)$ （按 4.1 节所述， $\beta_i(b)$ 不含位置 i 自身的状态得分，恰好补齐了这里"位置 $i+1$ 之后"的部分）。三者相乘再除以 $Z(x)$ 归一化，即得命题。■

一致性校验：对 b 求和应还原单点边缘概率 $P(y_{i-1} = a \mid x)$ ：

$$\sum_b \alpha_{i-1}(a) \exp(t(a, b) + s_i(b, x)) \beta_i(b) = \alpha_{i-1}(a) \underbrace{\sum_b \exp(t(a, b) + s_i(b, x)) \beta_i(b)}_{= \beta_{i-1}(a) \text{ (按 } \beta \text{ 的递推定义)}} = \alpha_{i-1}(a) \beta_{i-1}(a) \quad (20)$$

与 4.1 节的单点边缘概率公式完全吻合，说明推导自洽。

对应代码（0 起始索引，`i` 对应上式的 $i-1$ ，`i+1` 对应上式的 i ）：

```
pair = np.exp(alpha[i][:, None] + self.T + node_sc[i + 1][None, :] + beta[i + 1][None, :] - Z)
# pair[a, b] = P(y_i = a, y_{i+1} = b | x)
```

从边缘概率到最终梯度公式

状态特征权重 $W[f, y]$ 的梯度

状态特征只依赖当前位置的标签 y_i ，因此按第 3 节的展开公式，第 i 位对梯度的贡献就是单点边缘概率减去经验指示：

$$\frac{\partial \text{NLL}}{\partial W[f, y]} = \sum_{i: f \text{ 在位置 } i \text{ 激活}} \left[\underbrace{P(y_i=y|x)}_{\text{期望}} - \underbrace{\mathbb{1}\{y_i=y\}}_{\text{经验}} \right] \quad (21)$$

代入 4.1 节的公式 $P(y_i = y \mid x) = \alpha_i(y) \beta_i(y) / Z(x)$ ，对应代码：

```
marg = np.exp(alpha + beta - Z) # marg[i, y] = P(y_i=y|x)
for i in range(n):
    marg_i = marg[i].copy()
    marg_i[tag_ids[i]] -= 1.0 # 减去经验指示 1{y_i=y}
    for f in feat_ids[i]:
        if f >= 0:
            gradW[f] += marg_i
```

转移特征权重 $T[a, b]$ 的梯度

转移特征依赖相邻标签对 (y_{i-1}, y_i) ，因此需要 4.2 节的相邻两点边缘概率：

$$\frac{\partial \text{NLL}}{\partial T[a, b]} = \sum_{i=2}^n \left[\underbrace{P(y_{i-1}=a, y_i=b | x)}_{\text{期望}} - \underbrace{\mathbb{1}\{y_{i-1}=a, y_i=b\}}_{\text{经验}} \right] \tag{22}$$

对应代码（0 起始索引，`i` 从 0 到 $n-2$ ，对应 $i-1 \rightarrow i$ 的每一条相邻边）：

```
for i in range(n - 1):
    pair = np.exp(alpha[i][:, None] + self.T + node_sc[i + 1][None, :] + beta[i + 1][None, :] - Z)
    pair[tag_ids[i], tag_ids[i + 1]] -= 1.0      # 减去经验指示 1{y_i*=a, y_{i+1}*=b}
    gradT += pair
```

加上 L2 正则项

代码额外加了 L2 正则 $l2 \cdot (\|W\|^2 + \|T\|^2)$ ，其梯度是标准的 $2 \cdot l2 \cdot W$ 、 $2 \cdot l2 \cdot T$ ：

```
nll += l2 * (np.sum(self.W ** 2) + np.sum(self.T ** 2))
gradW += 2 * l2 * self.W
gradT += 2 * l2 * self.T
```

这部分与前向-后向算法无关，只是标准的正则化项梯度，附带列出以保证公式与代码完全对应。

公式与代码对照汇总

数学量	公式	代码对应
单点边缘概率	$P(y_i=y \mid x) = \frac{\alpha_i(y)\beta_i(y)}{Z(x)}$	<code>marg = np.exp(alpha + beta - Z)</code>
相邻边缘概率	$P(y_{i-1}=a, y_i=b \mid x) = \frac{\alpha_{i-1}(a) \exp(t(a, b) + s_i(b, x))\beta_i(b)}{Z(x)}$	<code>pair = np.exp(alpha[i][:,None] + T + node_sc[i+1][None, :] + beta[i+1][None, :] - Z)</code>
负对数似然	$\text{NLL} = \log Z(x) - \log \Psi(y, x)$	<code>nll = Z - gold_score</code>
状态权重梯度	$\frac{\partial \text{NLL}}{\partial W[f, y]} = \sum_i [P(y_i=y \mid x) - \mathbb{1}\{y_i=y\}]$ (在 f 激活的位置求和)	<code>marg_i[tag_ids[i]] -= 1.0; gradW[f] += marg_i</code>
转移权重梯度	$\frac{\partial \text{NLL}}{\partial T[a, b]} = \sum_i [P(y_{i-1}=a, y_i=b \mid x) - \mathbb{1}\{y_{i-1}=a, y_i=b\}]$	<code>pair[tag_ids[i], tag_ids[i+1]] -= 1.0; gradT += pair</code>

小结

1. 负对数似然的梯度一般形式是 "模型期望特征计数 - 训练数据经验特征计数"，这是指数族分布（对数线性模型）梯度的通用结论。
2. 要算出这个期望，需要把它拆解到每个位置（状态特征）和每条相邻标签边（转移特征）上，这就转化成了对单点边缘概率和相邻两点边缘概率的求解需求。
3. 单点边缘概率 $P(y_i = y \mid x) = \alpha_i(y)\beta_i(y)/Z(x)$ 、相邻边缘概率 $P(y_{i-1} = a, y_i = b \mid x) = \alpha_{i-1}(a) \exp(t(a, b) + s_i(b, x))\beta_i(b)/Z(x)$ ，两者都同时需要 α 与 β （除了序列末端的特殊情况），这正是线性链 CRF 训练必须执行完整的前向-后向算法、而不能只用前向算法的根本原因。